

AI-Based CV Shortlisting System

End-to-End Workflow & Technical Architecture Documentation

1. Executive Overview

The **AI-Based CV Shortlisting System** is an intelligent recruitment screening engine designed to evaluate, rank, and categorize candidate resumes against customized Job Descriptions (JDs). It integrates document parsing, text normalization, semantic matching, deterministic rule gates, and an optional local Large Language Model (LLM via Ollama) to generate explainable scores and shortlist reports.

2. End-to-End Pipeline Summary

Pipeline Stage	Key Responsibilities & Operations
1. Input Ingestion	Uploads Job Description & candidate CVs (PDF, DOCX, TXT) via the Streamlit interface.
2. Text Extraction & Cleaning	Extracts raw text streams (pypdf, pdfplumber, docx) and cleans noise & formatting.
3. JD Requirement Profiling	Extracts target seniority, required years, must-have skills, and degree prerequisites.
4. Candidate Feature Extraction	Parses educational degrees, graduation years, enrollment status, job titles, and skills.
5. Hard-Gate Knockout Check	Instantly filters corrupt files, candidates without required degrees, or invalid enrollment.
6. Multi-Component Scoring	Scores passing candidates across Skills (55%), Experience (22%), Education (13%), Title (10%).
7. Ranking & Categorization	Computes match percentage and segments into Highly Suitable, Suitable, Moderate, Not Suitable.
8. Results Dashboard & Export	Renders interactive Streamlit shortlists, LLM evidence summaries, and CSV downloads.

3. Detailed Component Architecture

3.1 Ingestion, Cleaning & Profiling (modules/extractor.py, cleaner.py, jd_profile.py)

- **Text Extraction:** Handles PDF, DOCX, and TXT with robust fallback mechanisms for corrupted or unstructured layouts.
- **Normalization:** Sanitizes formatting, normalizes technical compound terms (e.g. *Node.js*, *C++*, *CI/CD*), and removes noise.
- **JD Modeling:** Builds a structured profile extracting role title, seniority benchmarks (Entry: 0y, Junior: 1y, Mid: 3y, Senior: 5y, Lead: 8y), education requirements, and separated must-have vs. optional skill sets.

3.2 Candidate Parsing & Hard Gates (modules/education.py, experience.py)

- **Degree & Year Parsing:** Identifies highest degrees (BSc, MSc, PhD) and graduation years, determining if a candidate is graduated or currently enrolled.
- **Experience Calculation:** Measures domain-relevant tenure and strips out irrelevant academic coursework or student society listings.
- **Hard-Gate Knockouts:** Immediately routes unreadable files, missing required degrees, or enrollment mismatches into the **Rejected** category.

3.3 Multi-Component Scoring Model (modules/scoring.py & config.py)

Passing candidates are scored on a normalized scale across 4 components with dynamic weight adjustment:

Component	Weight	Scoring Formula & Logic
Skills Fit	55%	Must-Have skills count 2.5× higher than optional skills. Uses semantic similarity thresholding (Full credit ≥ 0.85 , floor credit at 0.70).

Experience Fit	22%	Non-linear shortfall curve evaluating domain relevance. For Entry/Trainee roles, fresh graduates are awarded top marks rather than penalized.
Education Fit	13%	Evaluates degree level and field of study relevance (Exact field = 100%, Allied Engineering = 80%, General Science = 60%).
Title Alignment	10%	Calculates token and semantic overlap between candidate's previous job titles and the target JD role family.

3.4 Ranking & Classification Bands (modules/ranker.py)

Tier	Score Range	Classification & Recruiter Action
Highly Suitable	75% – 100%	Primary candidate shortlist. Exceptional skill alignment & strong experience fit.
Suitable	55% – 74%	Recommended for HR interviews. Meets all core mandatory qualifications.
Moderate	35% – 54%	Borderline candidate. Minor skill overlaps or domain shifts; secondary review.
Not Suitable	0% – 34%	Fails to meet core skill and experience benchmarks.
Rejected	Hard Gate	Knocked out by mandatory prerequisite gates (unreadable file, no degree).

4. Optional Local LLM Reasoning Engine (modules/llm_engine.py)

In LLM mode (app_llm.py), the system connects to a local **Ollama** instance running models such as *llama3.2:1b*, *llama3.1:8b*, or *mistral* to enhance evaluation fidelity:

- **Tech Stack Decomposition:** Recognizes multi-part technologies (e.g., Mongo + Express + React + Node $\rightarrow$ MERN).
- **Contextual Employment Filtering:** Disregards student projects and extracurricular clubs from professional job counts.
- **Evidence Quoting:** Extracts and displays verbatim sentences from CVs justifying scores.
- **Recruiter Summaries:** Generates executive candidate summaries highlighting key strengths and areas of shortfall.

5. Key Deliverables & UI Features

- **Dual Dashboard Views:** Clean separation between Shortlisted candidates and Unsuitable / Rejected files.
- **Candidate Profiling Badges:** Visual tags indicating ■ New Grad, ■ Experienced, or ■ Undergraduate.
- **Dynamic Weight Renormalization:** Ensures candidates are never penalized for requirements absent from the JD.
- **ATS CSV Export:** One-click tabular export containing full candidate scores, metrics, and educational details.